Práctica 12 — Perseguir el cursor del ratón

Programación de Videojuegos · FP Lumara · Práctica previa: 06

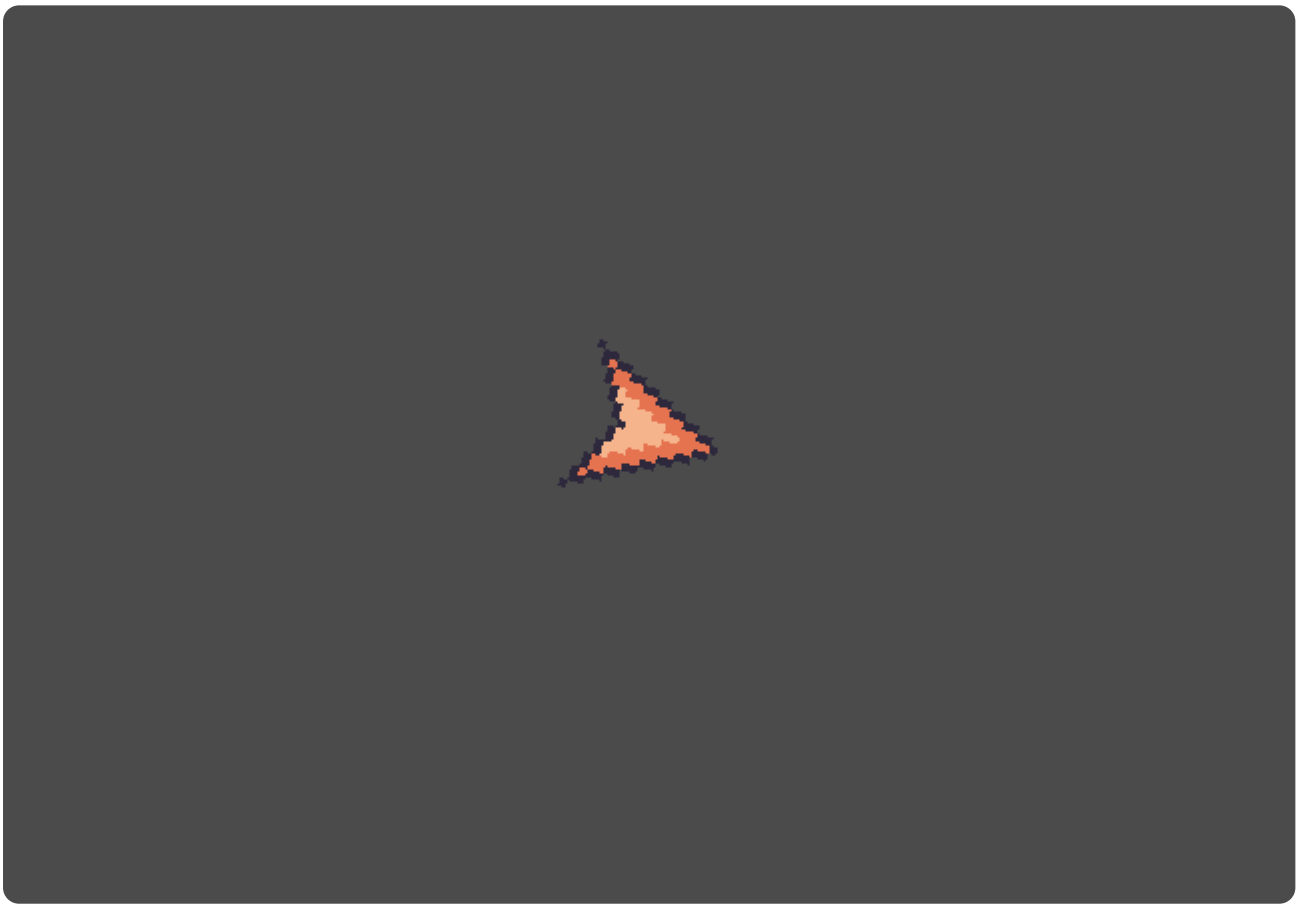
> *Lo que aprendes hoy: a calcular la dirección hacia un punto, a girar hacia él y a medir distancias.*

---

1. Qué vas a conseguir

Una nave que persigue al cursor del ratón por toda la pantalla. Apunta siempre hacia él, y cuando lo alcanza se para.

[CAPTURA 12.1]



“El resultado de la práctica 12: la nave siempre mira al ratón.”

---

2. Materiales

Descomprime `practica_12_inicio.zip`. En `assets/` hay una imagen nueva: `nave.png`.

Por qué hoy no usamos el personaje verde

Porque está dibujado **de frente**. Si lo giras para que mire al cursor, lo que ves es un muñeco tumbado boca abajo, no un personaje mirando a un sitio.

Una imagen que va a rotar tiene que estar dibujada **apuntando hacia un lado concreto**. En Godot, por convención, ese lado es **la derecha**: cuando la rotación de un nodo vale 0, se considera que mira hacia la derecha.

Ábrela en el Sistema de Archivos y comprueba que la punta de la nave apunta a la derecha.

> ***Esto no es un detalle de Godot: es una regla de todos los motores. Si un día dibujas tu propio personaje y quieres que rote, dibújalo mirando a la derecha.***

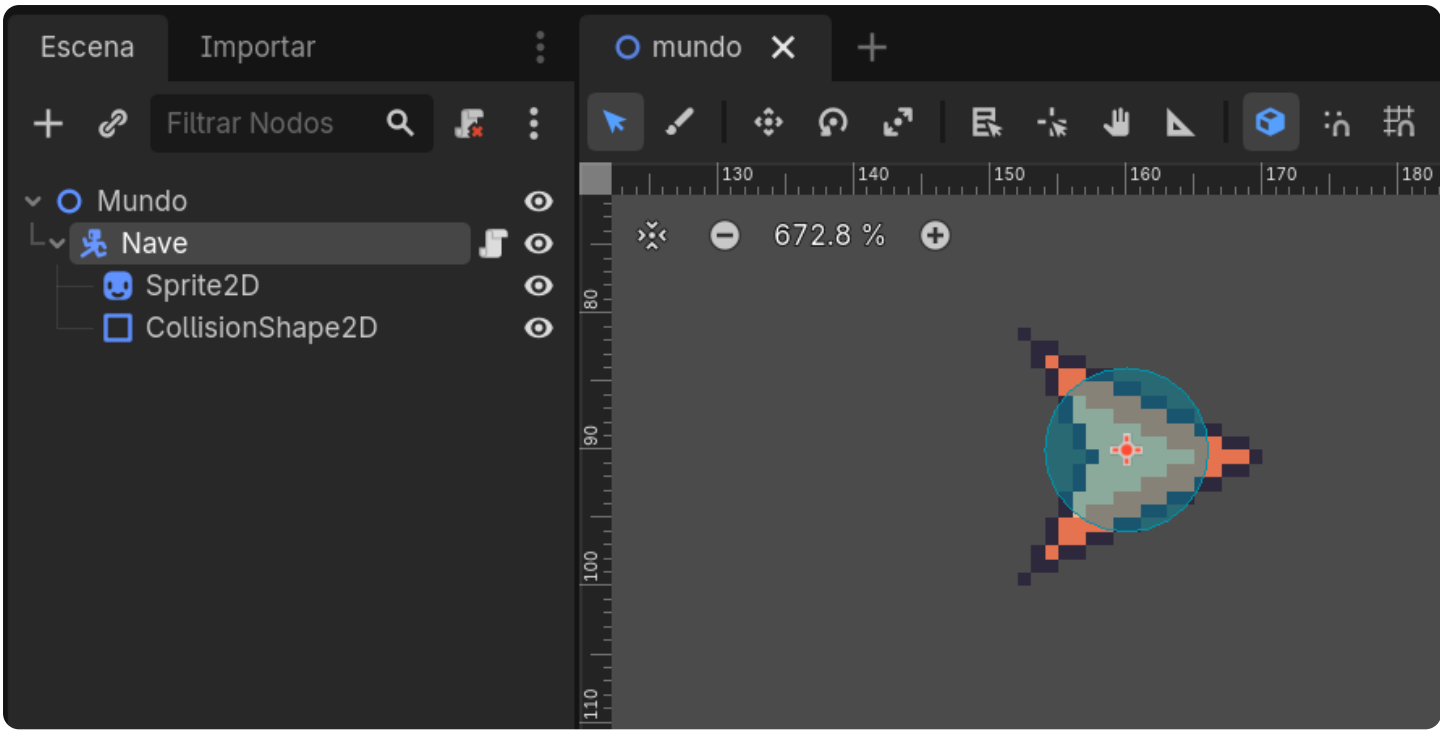
---

3. Paso a paso

1. Crea una **Escena 2D**, renombra el raíz a `Mundo`.
2. Añade un `CharacterBody2D` llamado `Nave`, en la posición `160, 90`.
3. Dentro de `Nave`, un `Sprite2D` llamado `Imagen` con la textura `nave.png`.
4. Dentro de `Nave`, un `CollisionShape2D` llamado `FormaColision` con una **Nuevo CircleShape2D** de radio `6`.

Es el mismo montaje de la práctica 06: cuerpo, imagen y forma.

[CAPTURA 12.2]



"Mismo montaje que el personaje: cuerpo, imagen y forma."

5. Añade a `Nave` un script en `res://scripts/nave.gd` :

```
extends CharacterBody2D

var velocidad := 120.0
var distancia_minima := 6.0


func _physics_process(_delta: float) -> void:
    var objetivo := get_global_mouse_position()
    look_at(objetivo)

    if global_position.distance_to(objetivo) > distancia_minima:
        velocity = global_position.direction_to(objetivo) * velocidad
    else:
        velocity = Vector2.ZERO

    move_and_slide()
```

6. Guarda como `escenas/mundo.tscn` y ejecuta con **F6**.

4. El código, línea a línea

`get_global_mouse_position()` Devuelve dónde está el cursor del ratón, en coordenadas del mundo. Cambia cada vez que el jugador mueve el ratón, así que hay que preguntarlo en cada fotograma.

Existe también `get_local_mouse_position()`, que lo da respecto al propio nodo. Nosotros queremos la global.

`look_at(objetivo)` Gira el nodo para que **su lado derecho** apunte al punto que le digas. Es la función que hace todo el trabajo de la rotación: no hay que calcular ningún ángulo a mano.

Aquí es donde se paga que la imagen esté dibujada mirando a la derecha.

`global_position.distance_to(objetivo)` Devuelve, en píxeles, lo lejos que está un punto de otro. Es la distancia en línea recta, la misma de Pitágoras que salió en la práctica 05.

`global_position.direction_to(objetivo)` Devuelve un vector **ya normalizado** que apunta del nodo hacia el objetivo. Es lo mismo que `Input.get_vector` te daba con el teclado, pero calculado hacia un punto en vez de leído de las teclas.

Fíjate en que con estas dos funciones tienes las dos preguntas resueltas: *¿hacia dónde?* y *¿cuánto queda?*.

`if ... else` **con** `distancia_minima` Sin esta comprobación, la nave nunca se estaría quieta: al llegar al cursor lo pasaría de largo, daría la vuelta, lo volvería a pasar, y acabaría **vibrando** encima de él para siempre.

Los 8 píxeles de margen son lo que en los videojuegos se llama *zona muerta*: un espacio donde se considera que ya has llegado.

`global_position` **y no** `position` `position` es la posición **respecto al padre**; `global_position`, respecto al mundo entero. Como el ratón se mide en coordenadas del mundo, hay que comparar peras con peras. Hoy `Nave` cuelga directamente de `Mundo`, que está en el origen, así que las dos valdrían: en cuanto haya jerarquías más profundas, no.

---

5. Pruébalo

- [ ] La nave apunta al cursor esté donde esté.
- [ ] Se mueve hacia él y lo alcanza.
- [ ] Al alcanzarlo **se para**, no vibra encima.
- [ ] Si mueves el ratón rápido, la nave se queda atrás y luego lo persigue.

- [ ] En el panel **Salida** no hay nada en rojo.

---

6. La teoría en cinco líneas

- `get_global_mouse_position()` dice dónde está el ratón.
- `look_at()` gira el nodo para que su derecha apunte a un punto.
- `direction_to()` da el vector normalizado hacia un punto.
- `distance_to()` da cuántos píxeles faltan.
- Sin una distancia mínima, un perseguidor vibra encima de su objetivo.

---

7. Si no te funciona

Lo que ves	Qué pasa	Cómo se arregla
La nave gira pero no avanza	Falta `move_and_slide()`	Va al final de la función
Vibra encima del cursor	Falta el `if` de la distancia mínima	Añádelo
Apunta hacia un lado raro	La imagen no está dibujada hacia la derecha	Usa `nave.png`, no el personaje
Gira como loca sin parar	Han puesto `look_at` con la posición local	Usa `get_global_mouse_position()`
No se mueve nada	El script está en `Mundo` o en `Imagen`	Va en el `CharacterBody2D`
Va lentísima o disparada	El valor de `velocidad`	Prueba entre 80 y 200
`Invalid call ... 'direction_to'`	Lo han llamado sobre el nodo, no sobre la posición	Es `global_position.direction_to(...)`

---

8. Retos

Verde · Haz que la nave vaya **más despacio cuando está cerca**, para que frene suavemente en vez de pararse de golpe. Pista: en vez de una velocidad fija, usa una que dependa de la distancia.

Amarillo · Haz lo contrario: una nave **cobarde** que huya del cursor. Solo hay que cambiar un signo o invertir la dirección. Piensa antes cuál de las dos cosas es.

Rojo · Que la nave gire **poco a poco** en vez de apuntar al instante, como si le costara girar. Pista: `look_at` pone la rotación de golpe. Investiga `rotate_toward()` o calcula el ángulo con `angle_to_point()` y acércate a él con `move_toward()` .

---

9. Autoevaluación

- [ ] Sé por qué una imagen que va a rotar debe dibujarse mirando a la derecha.
- [ ] Sé obtener la posición del ratón.
- [ ] Sé usar `look_at()` .
- [ ] Sé calcular la dirección y la distancia hacia un punto.
- [ ] Sé por qué hace falta una distancia mínima.
- [ ] Sé la diferencia entre `position` y `global_position` .

---

10. Vocabulario

Español	Inglés	Qué es
Posición global	Global position	Posición respecto al mundo
Mirar hacia	Look at	Girar para apuntar a un punto
Dirección hacia	Direction to	Vector normalizado hacia un punto
Distancia	Distance	Píxeles en línea recta entre dos puntos
Zona muerta	Dead zone	Margen en el que se considera que ya has llegado

---

11. Entrega

Comprime la carpeta como `apellido_practica12.zip` y súbela al Aula Virtual de Alexia.

Capturas necesarias en esta práctica

Código	Contenido
12.1	Resultado final, a ser posible en GIF
12.2	Árbol de tres nodos con la nave en el visor